

Grafana

Observabilidade e dashboards em tempo real, do dado bruto ao alerta.

Nicolas Motta

Prof. Diego R. Antunes • UTFPR

grafana.com

O QUE É

O Grafana não guarda dados. Ele se conecta onde os dados já estão e os transforma em painéis, gráficos e alertas.

Como o painel de um carro: os sensores medem, o painel mostra e acende a luz quando algo sai do normal.

Ficha técnica

Nome	Grafana
Site oficial	grafana.com
Criação	2014 · primeira versão pública em 19 de janeiro de 2014
Criador	Torkel Ödegaard
Empresa	Grafana Labs, fundada em 2014, sede em Nova York
Licença	Open source, AGPL-3.0
Feito em	Go (servidor) e TypeScript com React (interface)
Comunidade	cerca de 77 mil estrelas no GitHub

2013: métricas boas, tela ruim

O QUE O TIME USAVA

Graphite

Guardava as métricas da aplicação, mas montar dashboard e escrever query na tela dele era difícil e demorado.



A TELA QUE ELE QUERIA

Kibana 3

Dashboards interativos no navegador, fáceis de montar. Só que feitos para o Elasticsearch, não para métricas.



DEZ 2013

Grafana

Um fork do Kibana 3 com editor de queries do Graphite: a tela do Kibana para séries temporais.

A ideia que ficou: separar a tela do armazenamento. O Grafana nunca guardou as métricas, ele conversa com quem guarda. Por isso hoje lê Prometheus, SQL, Loki e dezenas de outras fontes.

De projeto paralelo a empresa

CRIADOR

Torkel Ödegaard

Desenvolvedor e consultor. Fez o primeiro commit num fork do Kibana e lançou a v1.0 em 19/01/2014. Segue na Grafana Labs até hoje.

EMPRESA

Grafana Labs

Em 2014, em Nova York, Torkel se junta a Raj Dutt e Anthony Woods na raintank, a empresa que virou a Grafana Labs.

Open core

O Grafana é grátis; a empresa vende Cloud, Enterprise e suporte

400 mil+

instalações ativas do Grafana em 2019

US\$ 600 mi+

de receita anual recorrente em 2026

De um fork a um padrão de mercado

Dez 2013	Jan 2014	2015	2016	2020	2021	2025-26
Fork do Kibana 3 para ler o Graphite	v1.0 pública, um app que rodava todo no navegador	v2.0: servidor em Go, usuários e banco próprio	v4.0: alertas dentro do Grafana	v7.0: data frames e transformações	v8.0: alertas unificados, uPlot e licença AGPL-3.0	v12 e v13: dashboards como código, Git Sync

Hoje: a pilha LGTM da Grafana Labs (Loki para logs, Grafana, Tempo para traces e Mimir para métricas). A demo roda o Grafana 13.2.

Observabilidade e visualização de dados

Usado por times de DevOps, SRE, dados e IoT para saber o que está acontecendo, agora.



Métricas

Números ao longo do tempo: CPU, vendas, temperatura de um tanque.

Prometheus · Mimir



Logs

O registro em texto de cada evento que um sistema produz.

Loki · Elasticsearch



Traces

O caminho de uma requisição passando por vários serviços.

Tempo · Jaeger

QUEM USA EM PRODUÇÃO

10 mil+

empresas clientes da Grafana Labs,
fora quem usa só a versão gratuita

Anthropic

Heineken

NVIDIA

Microsoft

Salesforce

Uber

Siemens

LATAM Airlines

0 cliente percebia a falha antes do time

75 mi+

passageiros por ano

500+

serviços no sistema digital

~40

serviços acionados por um clique no app

Antes do Grafana Cloud

- Dados de monitoramento chegavam com horas de atraso
- Clientes reclamavam de erros no site e no app antes do time notar
- Alertas demais: o plantão não sabia o que era urgente

0 que mudou

- Métricas em tempo real e SLOs medidos pela experiência do cliente
- Grafana IRM para organizar a resposta a incidentes

Os resultados

87%

menos tempo para detectar incidentes graves

61%

menos alertas ruidosos para o plantão

23%

menos tempo para reparar (meta: 50%)

"A observabilidade de verdade precisa começar pelos clientes."

Carlos Hernandez Saavedra, Head of Cloud & SRE da LATAM (tradução livre)

O que dá para fazer



Dashboards

Gráficos, medidores, tabelas, mapas e números



Consultas

SQL, PromQL ou LogQL escritos no painel



Variáveis

Filtros no topo que mudam todos os painéis



Alertas

Regras, estados e notificações por canal



Anotações

Marcam eventos importantes nos gráficos



Explore

Investiga dados sem montar um painel



Provisionamento

Configuração como código, versionada no Git



Times e plugins

Permissões por equipe e catálogo de plugins

Fontes de dados que já vêm prontas

Métricas

- Prometheus
- Graphite
- InfluxDB
- Mimir

Bancos SQL

- PostgreSQL (na demo)
- MySQL
- Microsoft SQL Server

Logs e traces

- Loki
- Elasticsearch
- Tempo
- Jaeger

Nuvem

- Amazon CloudWatch
- Azure Monitor
- Google Cloud Monitoring

E o catálogo de plugins adiciona outras, como MongoDB, Snowflake e APIs em JSON ou CSV.

Três jeitos de rodar

GRÁTIS • USADO NA DEMO

Grafana OSS

Open source, você hospeda. Roda em Linux, Windows, macOS, Docker e Kubernetes (Helm).

PAGO

Enterprise

Você hospeda, com suporte, relatórios, permissões mais finas e plugins exclusivos.

TEM PLANO GRÁTIS

Grafana Cloud

Serviço gerenciado pela Grafana Labs, já com Loki, Mimir e Tempo. A LATAM usa este.

O Grafana guarda usuários e dashboards num banco próprio: SQLite por padrão, MySQL ou PostgreSQL em produção.

COMO FUNCIONA

Consulta na hora, desenha na tela



Fontes de dados

Prometheus, PostgreSQL, Loki, CloudWatch... Os dados ficam onde já estão.



Grafana

Roda a query de cada painel, transforma o resultado e avalia as regras de alerta.



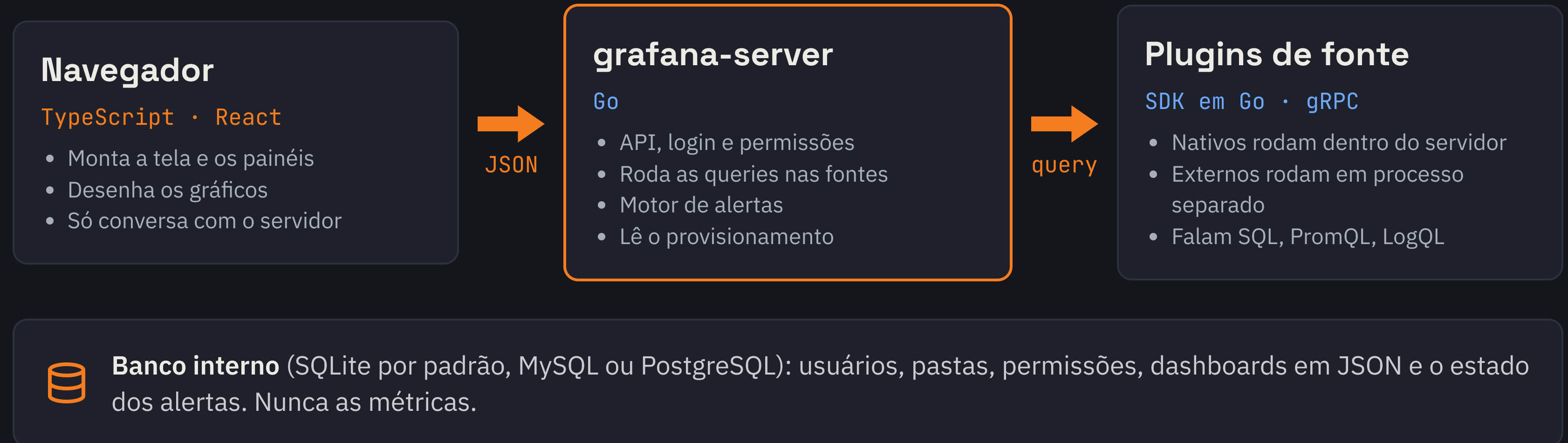
Dashboards

no navegador, ao vivo

Alertas

Slack, Teams, e-mail, webhook

Navegador, servidor e plugins



Cinco passos a cada refresh

1

Monta a query

Troca as variáveis pelo que está escolhido na tela

navegador

2

Executa

O servidor expande as macros e roda a query na fonte

servidor

3

Data frame

O resultado vira uma tabela interna, com tipo por coluna

servidor

4

Processa

Separa em séries, reduz a um valor, aplica unidade e cores

navegador

5

Desenha

A visualização escolhida pinta: linha, número, medidor, tabela

navegador

O navegador nunca fala com o banco. Quem conecta na fonte de dados é o servidor do Grafana. A senha fica lá e o banco nem precisa estar exposto.

Um SQL, qualquer período e filtro

Escrito no painel

```
WHERE $__timeFilter(momento)
      AND linha IN ($linha)
```



O que chega ao banco

```
WHERE momento BETWEEN '13:00' AND '14:00'
      AND linha IN ('L1','L2')
```

`$__timeFilter`

Vira o período do seletor de tempo.
Mudou para 15 minutos, todo painel muda junto.

`$__interval`

Tamanho do balde de agrupamento,
calculado pelo período e pela largura do painel.

`$linha`

Variável: o filtro no topo do dashboard.
A lista vem de uma query e aceita várias opções ou "All".

O painel pede: um POST com JSON

Painel "Temperatura dos tanques" pedindo dados ao servidor

```
POST /api/ds/query
{
  "from": "now-30m", "to": "now",
  "queries": [{
    "refId": "A",
    "datasource": { "uid": "cervejaria-pg" },
    "format": "time_series",
    "rawSql": "SELECT $__timeGroupAlias(...) ...",
    "maxDataPoints": 918,
    "intervalMs": 5000
  }]
}
```

maxDataPoints: 918

A largura do painel em pixels. Não adianta trazer mais pontos do que a tela mostra.

intervalMs: 5000

Vira o `$__interval`: período dividido pelos pontos, nunca abaixo do mínimo da fonte (5 s).

refId: A

Cada query do painel tem uma letra. A resposta volta separada por ela.

O servidor responde: um data frame

Resposta real da demo, resumida

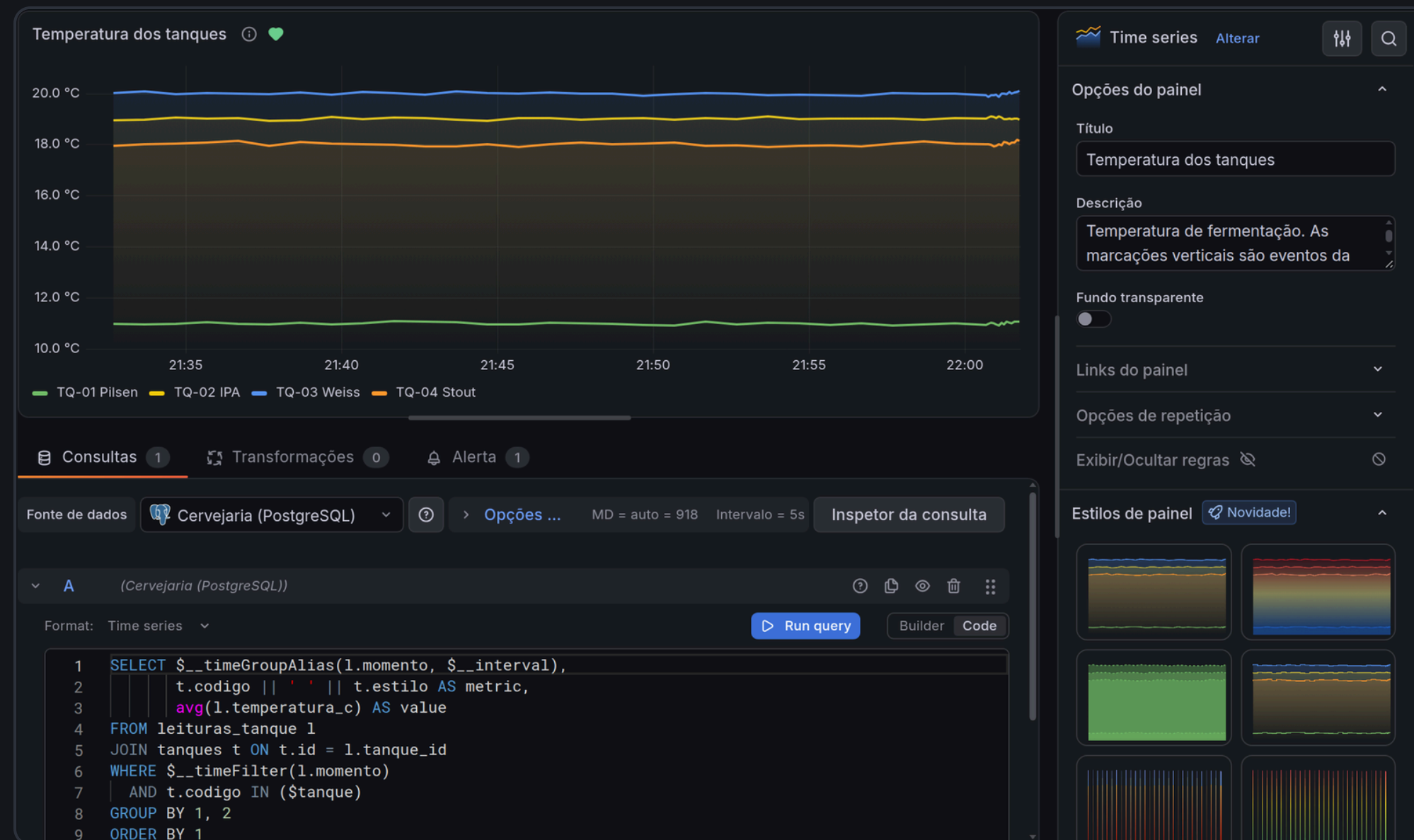
```
"results": { "A": { "frames": [{
  "schema": {
    "meta": { "type": "timeseries-wide" },
    "fields": [
      { "name": "Time", "type": "time" },
      { "name": "TQ-02 IPA", "type": "number" } ] },
  "data": { "values": [
    [1791334720000, 1791334780000, ...],
    [18.97, 19.03, ...] ] }
}] } }
```

Time	TQ-02 IPA
21:58:40	18.97
21:59:40	19.03
22:00:40	19.01

Colunar: cada campo é um vetor com tipo. A coluna **metric** do SQL virou o nome da série.

Formato único: todo plugin devolve data frames, seja SQL, Prometheus ou Loki. Por isso qualquer painel desenha qualquer fonte.

O navegador desenha num canvas



uPlot

Biblioteca que desenha o painel Time series desde o Grafana 7.4 (2021), no lugar do antigo jQuery Flot. Pinta em canvas, não em SVG: milhares de pontos sem travar.

fieldConfig + options

Antes de pintar, aplica unidade (°C), decimais, cores, legenda e tooltip.

MD = auto = 918

O editor mostra os pontos e o intervalo do pedido anterior.

Do número à informação

Redução

20.4, 20.9, 21.3
→ **21.3**

Stat e gauge mostram um valor só: aqui, o último

Unidade

21.3
→ **21,3 °C**

Celsius, porcentagem, bar:
o Grafana formata

Thresholds

2% **3%** **5%**

Faixas de valor viram cor

Value mapping

0 → **DESLIGADA**

Troca valor por texto e cor

A query só traz números. Unidade, cor, texto e tipo de gráfico são decisões do painel: o mesmo resultado vira linha, medidor ou tabela sem mudar o SQL.

Todo dashboard é um JSON

grafana/dashboards/cervejaria.json, resumido

```
{
  "uid": "cervejaria",
  "title": "Cervejaria Byte: Monitoramento...",
  "time": { "from": "now-1h", "to": "now" },
  "refresh": "5s",
  "templating": { "list": [ /* $tanque */ ] },
  "annotations": { "list": [ /* eventos */ ] },
  "panels": [ /* 10 painéis */ ],
  "schemaVersion": 41,
  "version": 1
}
```

Salvar é gravar o JSON

Fica no banco interno, e cada salvamento vira uma versão.

Exportar e importar

Copiou o JSON, colou em outro Grafana: mesmo dashboard.

Como código

Na demo o Grafana lê este arquivo do disco. O Grafana 12 trouxe o Git Sync.

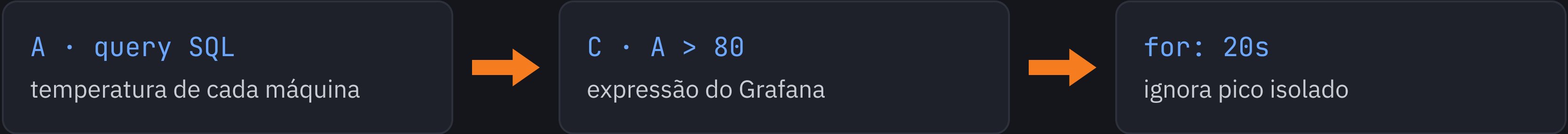
Anatomia de um painel

Painel 5, "Temperatura dos tanques", resumido

```
{
  "type": "timeseries", // 1
  "title": "Temperatura dos tanques",
  "datasource": { "uid": "cervejaria-pg" }, // 2
  "targets": [{ "refId": "A", // 3
    "rawSql": "SELECT $__timeGroupAlias(...)" }],
  "fieldConfig": { "defaults": { // 4
    "unit": "celsius", "decimals": 1,
    "custom": { "lineWidth": 2, "fillOpacity": 10 } } },
  "options": { "tooltip": { "mode": "multi" } },
  "gridPos": { "x": 0, "y": 5, "w": 16, "h": 10 } // 5
}
```

- 1 type:** qual visualização desenha (timeseries, stat, gauge, table)
- 2 datasource:** qual fonte, só pelo uid. Sem endereço nem senha
- 3 targets:** as consultas, com macros e variáveis
- 4 fieldConfig:** unidade, cores, thresholds e mappings
- 5 gridPos:** lugar numa grade de 24 colunas; altura em blocos de 30 px

Uma regra, quatro alertas



Cada linha do resultado vira uma instância

Label	Valor	Estado
maquina=1	72 °C	Normal
maquina=2	86 °C	Disparado
maquina=3	69 °C	Normal
maquina=4	75 °C	Normal

Estados de uma instância

- Normal** condição falsa
- Pendente** verdadeira, contando o for
- Disparado** manda a notificação

Roda no servidor, com o dashboard aberto ou fechado.
Também existem NoData e Error.

Do alerta à mensagem



Tipos de contact point

Slack

Teams

E-mail

Telegram

PagerDuty

Webhook

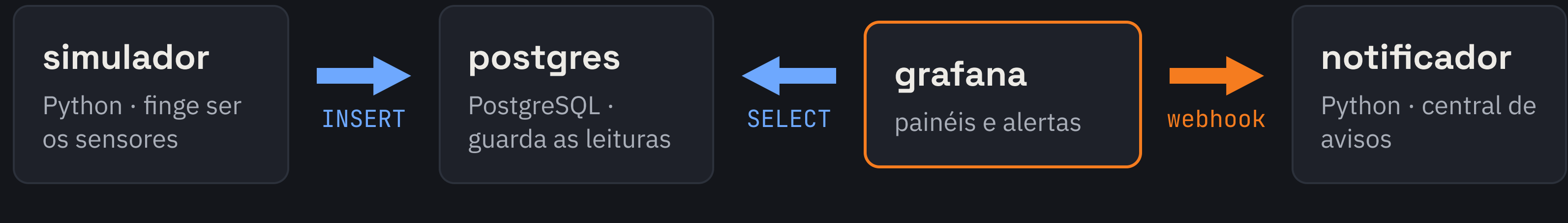
Mensagem com template

```
summary: Máquina {{ $labels.maquina }}  
está a {{ $values.A }} °C
```

DEMONSTRAÇÃO

Cervejaria Byte: os conceitos na prática

o que vai rodar no localhost



4 tanques de fermentação e 2 linhas de envase. Leituras a cada 5 segundos e alerta quando um tanque passa do limite.

Do sensor ao alerta, no código

1 • Sensor

`simulador.py`

```
cur.executemany(  
    "INSERT INTO  
    leituras_tanque  
    (momento,  
    temperatura_c,  
    ...)", leituras)
```

Grava cada tanque a cada 5 s

2 • Banco

`01_schema.sql`

```
CREATE TABLE  
leituras_tanque (  
    momento  
        TIMESTAMPTZ,  
    tanque_id INT,  
    temperatura_c  
        NUMERIC, ...)
```

Série temporal num PostgreSQL comum

3 • Painel

`cervejaria.json`

```
SELECT  
    $__timeGroupAlias(  
        momento,  
        $__interval), ...  
WHERE  
    $__timeFilter(...)   
AND ... ($tanque)
```

SQL com macros e a variável

4 • Alerta

`alertas.yml`

```
interval: 10s  
condition: C  
    # excesso > 0  
for: 20s  
type: webhook  
url: http://  
    notificador:8000
```

Roda no servidor e chama o webhook



0 DASHBOARD

1 **Refrigeração:** 0 e 1 do banco viram Ligada e DESLIGADA

2 **Temperatura:** as linhas tracejadas são anotações de eventos

3 **Desvio do ideal:** medidores mudam de cor pelos limites

4 **Alertas ativos:** a regra disparou para o TQ-02

OBRIGADO

Perguntas?

Código da demo

github.com/nicolassmotta/grafana- cervejaria

Documentação

grafana.com/docs

Site oficial

grafana.com

Testar sem instalar

play.grafana.org